

Fundamentos de IA y Python

Unidad 1 — Conceptos, herramientas y bases de Python para el curso.

Contenido

01 Introducción a la IA, herramientas y entorno

02 Trabajando con notebooks

03 Python básico

04 Python intermedio

05 Programación asíncrona

06 Validación de datos con Pydantic

01

**Introducción a la IA,
Herramientas y Entorno**

¿Qué es la Inteligencia Artificial?

A lo largo de la historia se ha definido la IA desde perspectivas distintas, lo que ha generado enfoques distintos en su desarrollo.

“La IA es la ciencia y la ingeniería de hacer máquinas inteligentes”

— McCarthy, 1956

“La IA es el estudio de agentes que perciben su entorno y toman acciones para maximizar su probabilidad de éxito”

— Russell & Norvig, 2021

No hay una única definición "correcta" — cada una enfatiza algo distinto: simular inteligencia vs. actuar racionalmente en un entorno.

Tipos de Inteligencia Artificial

Concepto	Descripción
IA	Simula procesos cognitivos humanos: aprendizaje, razonamiento, decisión
Machine Learning	Aprende de datos y mejora su rendimiento sin ser programado explícitamente
Deep Learning	Usa redes neuronales profundas para modelar patrones complejos
LLM	Modelos entrenados con grandes cantidades de texto para lenguaje natural
GPT	Modelos que generan texto mediante redes neuronales entrenadas en texto
IA Generativa	Crea contenido nuevo (texto, imágenes, música) a partir de patrones aprendidos

Lenguajes de programación para IA

Lenguaje	Para qué se usa en IA
Python	Versátil, fácil de aprender — TensorFlow, PyTorch, scikit-learn
R	Análisis estadístico y visualización — ciencia de datos
Java	Robusto y portable — aplicaciones empresariales de IA, Weka
C++	Rápido y eficiente — sistemas de IA de alto rendimiento y juegos

Este curso trabaja íntegramente en Python — el lenguaje dominante en IA y ciencia de datos.

Frameworks y entornos de desarrollo

Frameworks

TensorFlow	De Google, flexible para redes neuronales y deep learning
PyTorch	Popular por su facilidad de uso — ideal para investigación
scikit-learn	Biblioteca de Python para ML clásico, simple de usar
Keras	API de alto nivel sobre TensorFlow

Entornos

Anaconda	Gestión de entornos y paquetes de Python
uv (Astral)	Alternativa más ligera y rápida a Anaconda/conda
VS Code	Editor ligero con extensiones para IA (p. ej. Copilot)
Jupyter	Entorno interactivo para ciencia de datos
Google Colab	Notebooks en la nube, sin instalación local

Instalación del entorno de trabajo

El curso usa Python 3.12+ dentro de un entorno virtual (aislado del resto del sistema)

Dos rutas equivalentes, cubiertas paso a paso en 1_Introducción.ipynb

- Anaconda: plataforma completa con conda para crear y gestionar entornos
- uv (Astral): más ligero y rápido, gestiona entornos y dependencias con un solo binario

Ambas rutas terminan igual: un entorno con Python + un kernel de Jupyter listo para los notebooks del curso

Los comandos exactos (conda/uv, por sistema operativo) están en el notebook — esta diapositiva resume el porqué, no el cómo.

02

Trabajando con Notebooks

¿Qué es un notebook y cómo se estructura?

Un notebook combina código ejecutable, texto explicativo y resultados en un mismo documento

Se usa en VS Code o Jupyter, seleccionando el entorno (kernel) que se creó en la Unidad 1.

- Celdas de código: se ejecutan con Shift + Enter
- Celdas de markdown: texto, títulos e imágenes

El resultado de la última instrucción de una celda se muestra automáticamente debajo de ella

El orden de ejecución importa

Las celdas no tienen que ejecutarse en el orden en que aparecen en la página — y eso puede generar resultados inesperados.

- La primera vez imprime "bananas"
- Si se reejecuta la celda de impresión sin correr las anteriores, el resultado cambia
- El notebook recuerda el estado de las variables, no el orden visual

```
fruta_favorita = "bananas" # ... más abajo en el notebook ...  
fruta_favorita = f"cualquier cosa menos {fruta_favorita}"  
  
# celda de impresión, ejecutada dos veces:  
print(f"Mi fruta favorita es {fruta_favorita}")
```

El kernel

- El kernel es el proceso de Python que ejecuta el código del notebook y guarda su estado (variables, imports)
- Reiniciar el kernel borra todas las variables definidas — hay que volver a ejecutar las celdas necesarias
- Un kernel "colgado" o con estado confuso se soluciona casi siempre reiniciándolo

Herramientas complementarias

- Comandos mágicos de IPython: %ls, %pwd — funcionan igual en Windows, macOS y Linux
- Comandos de shell con !: !pip install, etc.
- tqdm agrega una barra de progreso a un bucle largo
- IPython.display.Markdown muestra texto con formato enriquecido como salida de una celda

```
from tqdm import tqdm

for i in tqdm(range(100)):
    ...

# barra de progreso en vez de esperar a ciegas
```

03

Python Básico

Variables y tipos de datos

- Python no exige declarar el tipo — se infiere del valor asignado
- str, int, float, bool son los tipos básicos más usados
- `type(variable)` muestra el tipo real de cualquier valor

```
nombre = "Ana"           # str (texto)
edad = 21                 # int (entero)
promedio = 8.7            # float (decimal)
es_estudiante = True     # bool (verdadero/falso)

print(f"{nombre} tiene {edad} años")
```

Estructuras de datos

Estructura	Ejemplo	Característica
Lista (list)	["manzana", "banana"]	Ordenada y modificable
Tupla (tuple)	(4.6, -74.0)	Ordenada e inmutable
Diccionario (dict)	{"nombre": "Carlos"}	Pares clave-valor
Conjunto (set)	{"Python", "R"}	Sin duplicados, sin orden

Elegir la estructura correcta simplifica el resto del código: listas para secuencias, dict para búsquedas por clave.

Control de flujo

- if / elif / else: ejecuta un bloque según una condición
- for: recorre los elementos de una colección uno por uno
- while: repite mientras una condición se mantenga verdadera

```
if temperatura > 25:
    print("Hace calor")
elif temperatura > 15:
    print("Clima templado")
else :
    print("Hace frío")

for fruta in frutas:
    print(fruta)
while contador < 5:
    contador += 1
```

Excepciones y funciones

- try/except evita que un error detenga todo el programa — se maneja el caso esperado
- Las funciones (def) evitan repetir código: se definen una vez y se reutilizan con distintos argumentos

```
try :  
    resultado = 10 / 0  
except ZeroDivisionError:  
    print("No se puede dividir entre cero")  
  
def saludar (nombre_persona):  
    return f"¡Hola, {nombre_persona}!"
```

04

Python Intermedio

Comprensiones de listas y diccionarios

- Una comprensión construye una lista o diccionario nuevo en una sola línea, a partir de otro iterable
- Se puede filtrar con if dentro de la misma expresión
- Más compacto y, para la mayoría, más legible que el bucle equivalente

```
fruits = ["Apples", "Bananas", "Pears"]

# lista nueva, sin bucle for explícito:
shouted = [f.upper() for f in fruits]

# también con diccionarios, y con filtro:
mapping = {
    f: f.upper()
    for f in fruits if not f.startswith("A")
}
```

Generadores

- yield convierte una función en un generador:
produce valores uno a la vez, sin guardar toda la secuencia en memoria
- Útil para secuencias muy largas o infinitas
- yield from delega en otro generador o iterable

```
def number_generator ():  
  
    for i in range (1, 6):  
        yield i          # "pausa" y entrega un valor a la vez  
gen = number_generator() next(gen)  
# 1 next(gen)  
# 2 (continúa donde se quedó)
```

Type hints y subclasses

- Los type hints (str, Optional[str], Union[...]) documentan qué tipo se espera, sin forzarlo en tiempo de ejecución
- Una subclase hereda atributos y métodos de su clase padre y puede agregar o sobrescribir los propios

```
from typing import Optional
class Libro :
    titulo: str
    autor: Optional[str] =
None
class LibroDigital (Libro):
    formato: str = "epub" # hereda de Libro y agrega un campo
```

Decoradores

- Un decorador envuelve una función para agregarle comportamiento sin modificar su código interno
- `@classmethod` y `@staticmethod` son decoradores incluidos en Python
- Este patrón se usa para registrar herramientas de un agente (`@function_tool` en Unidad 5)

```
def medir_tiempo (func):  
  
    def envoltura (*args, **kwargs):  
        inicio = time.time()  
        resultado = func(*args, **kwargs)  
  
        print(f"Tardo: {time.time()-inicio:.2f}s" )  
  
        return resultado  
  
    return envoltura @medir_tiempo def tarea_lenta ():  
        ...
```

05

Programación Asíncrona

Corrutinas y await

- `async def` define una función asíncrona (una corrutina)
- Llamarla sin `await` solo crea el objeto corrutina — no ejecuta nada
- `await` la ejecuta y cede el control mientras espera (p. ej. una llamada de red)

```
async def do_some_work ():
```

```
await asyncio.sleep(1)
```

```
return "listo"
```

```
# do_some_work()          # NO ejecuta la corrutina, solo la crea  
await do_some_work()      # así sí se ejecuta y se espera el  
resultado
```

Secuencial vs. paralelo

- Encadenar `await` una tras otra sigue siendo secuencial — no gana nada en tiempo
- `asyncio.gather()` lanza varias corrutinas a la vez y espera a que todas terminen
- No es multithreading real: es concurrencia en un solo hilo, aprovechando los tiempos de espera

```
# secuencial: espera una tarea, luego la otra
await tarea_1()

await tarea_2() # en paralelo: ambas corren a la vez

await asyncio.gather(tarea_1(), tarea_2())
```

Generadores asíncronos: simulando streaming

- Combina `async def` con `yield`: produce valores uno a la vez, de forma asíncrona
- Así es como, en la práctica, un LLM "transmite" su respuesta token por token

```
async def stream_tokens (texto):  
  
    for palabra in texto.split():  
  
        await asyncio.sleep(0.1)  
  
        yield palabra  
    async for token in stream_tokens("Hola desde el LLM") :  
  
        print(token)
```

06

Pydantic:
Validación de Datos

¿Qué es Pydantic y cuándo usarlo?

Pydantic define modelos de datos con tipos, y valida automáticamente que los datos recibidos los cumplan

Validar datos de entrada de una API

Definir configuración de una aplicación

Estructurar y limpiar la salida de un LLM

Definir, validar y serializar

- BaseModel define la forma esperada de los datos con anotaciones de tipo normales de Python
- Si los datos no cumplen el modelo, Pydantic lanza un `ValidationError` claro y específico
- `model_dump()` / `model_dump_json()` serializan el objeto de vuelta a dict o JSON

```
from pydantic import BaseModel

class Usuario (BaseModel):
    nombre: str
    edad: int

u = Usuario(nombre=
    "Ana", edad="21")

# "21" se convierte a int
u.model_dump()      # -> dict de Python

u.model_dump_json() # -> string JSON
```

Validación avanzada

- Field() agrega restricciones (gt, min_length, ...) y metadatos a un campo
- @field_validator define una regla de validación personalizada para un campo específico
- model_config controla el comportamiento general del modelo (p. ej. si acepta campos extra)

```
from pydantic import Field, field_validator

class Producto (BaseModel):
    precio: float = Field(gt=0)  # debe ser mayor que 0

@field_validator("precio" )
    def redondear (cls, v):
    return round(v, 2)
```

Caso práctico: limpiar la salida de un LLM

- La salida de un LLM en JSON puede traer tipos incorrectos ("90" en vez de 90) o campos que sobran
- Pydantic convierte "90" a int automáticamente e ignora el campo extra, devolviendo un objeto listo para usar

```
class Ingrediente (BaseModel):  
    nombre: str  
    cantidad: float  
    unidad: str  
  
class Receta (BaseModel):  
    nombre_receta: str  
    ingredientes: List[Ingrediente]  
    tiempo_preparacion_min: int  
  
receta = Receta.model_validate_json(salida_llm)
```

Resumen de la Unidad 1

Tema	Notebook del curso
IA, herramientas e instalación	Unidad 1/1_Introducción.ipynb
Notebooks en VS Code / Jupyter	Unidad 1/fundamentos/1_notebooks.ipynb
Python básico	Unidad 1/fundamentos/2_python_basics.ipynb
Python intermedio	Unidad 1/fundamentos/3_python_intermedio.ipynb
Programación asíncrona	Unidad 1/fundamentos/4_async_python.ipynb
Validación de datos con Pydantic	Unidad 1/fundamentos/5_pydantic.ipynb

¡Gracias!

Estas bases de Python y del entorno de trabajo se usan en todas las unidades del curso.